ENCAPSULAMIENTO, HERENCIA y CLASES ABSTRACTAS.

Programador Universitario en Sistemas
Primer Año - 2° cuatrimestre

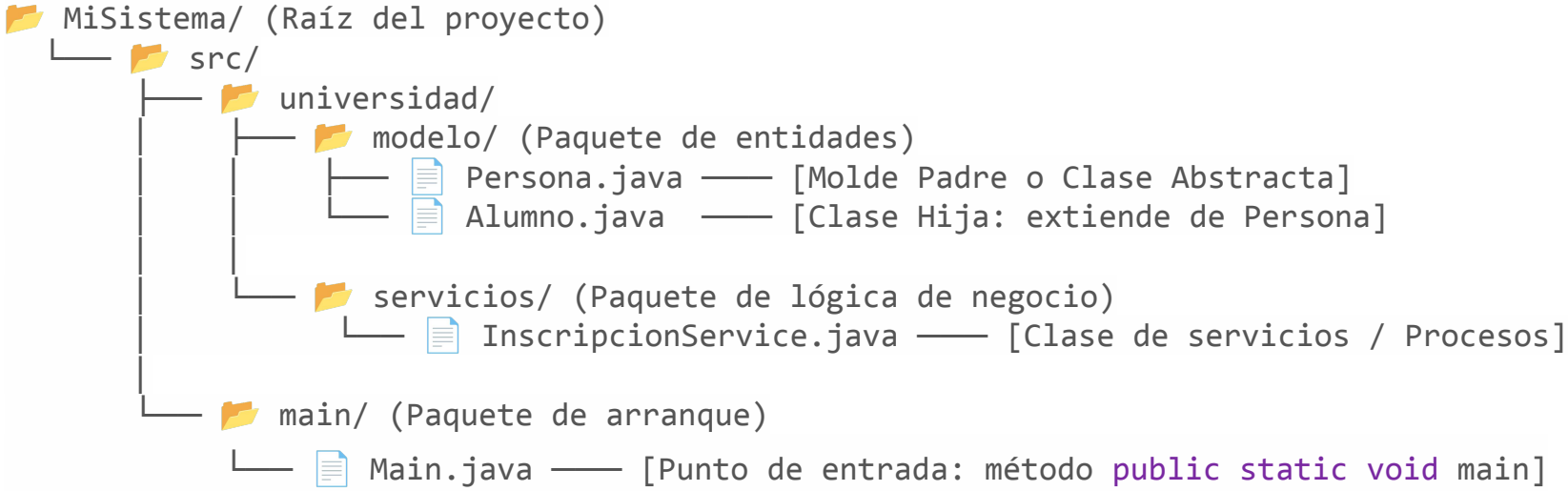
De las clases y objetos a la herencia y la abstracción

En esta clase vamos a trabajar:

- Organización de un proyecto Java.
- Clases y paquetes.
- Modificadores de acceso.
- Encapsulamiento.
- Herencia.
- extends.
- super.
- Clases abstractas.
- Métodos abstractos.
- Interfaces.

¿Dónde está nuestro código?

Un proyecto Java puede organizarse de la siguiente manera:



¿Qué es un package?

En Persona.java podemos escribir:

```
package universidad.modelo;

public class Persona {

}
```

La primera línea:

```
package universidad.modelo;
```

indica que Persona pertenece al paquete:

universidad.modelo

¿Qué significa public class Persona?

```
public class Persona { }
```

Tenemos dos conceptos diferentes:

`public class` y `class Persona`

class Persona: Define una clase llamada Persona.

public: Indica que la clase puede ser accesible desde otros paquetes.

Si Persona está en: universidad.modelo

y Main está en: universidad.main

¿Puede Main utilizar Persona si Persona es public?

```
import universidad.modelo.Persona;
```

¿Dónde se utiliza Persona?

Tenemos nuestra clase:

Persona.java

```
package universidad.modelo;

  public class Persona { private String
nombre;

        public Persona(String nombre)
              { this.nombre = nombre; }

        public String getNombre()
              { return nombre; }
  }
```

Main.java

```
package universidad.main;

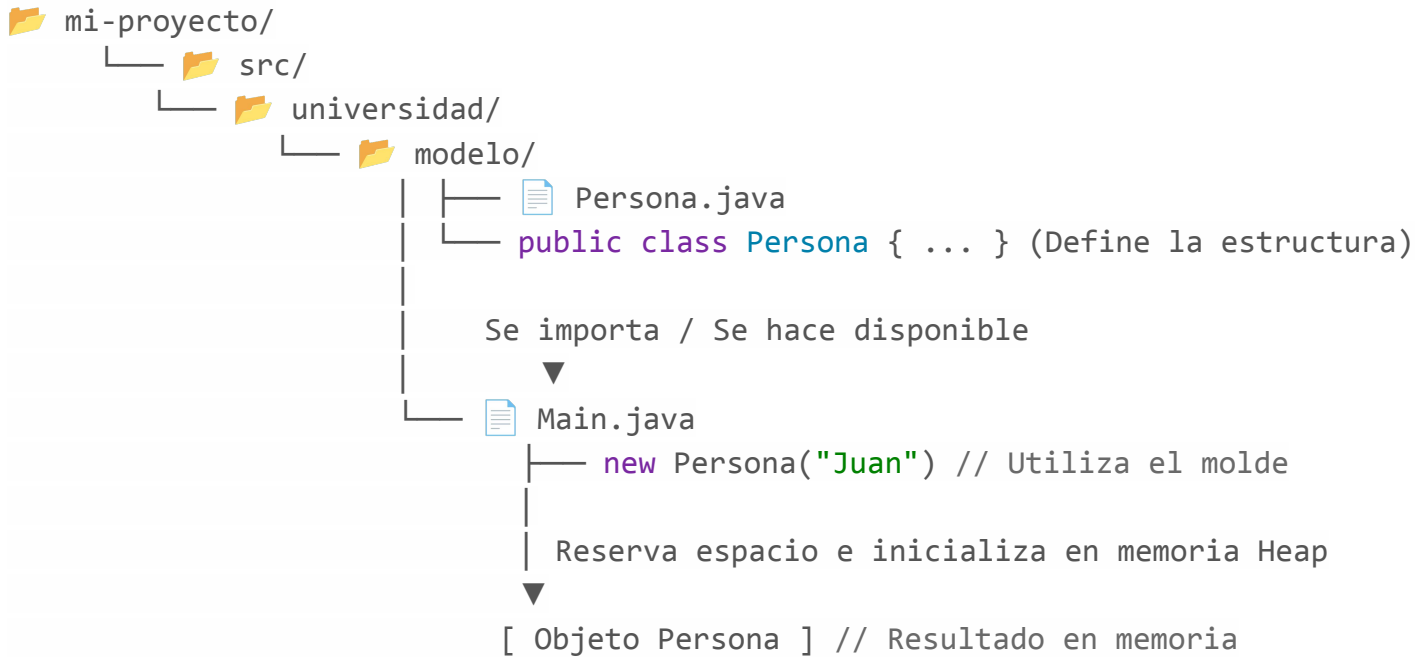
import universidad.modelo.Persona;

  public class Main { public static void
main(String[] args) {

     Persona persona1 = new Persona("Juan");
System.out.println(persona1.getNombre()); }
  }
```

Recorrido completo

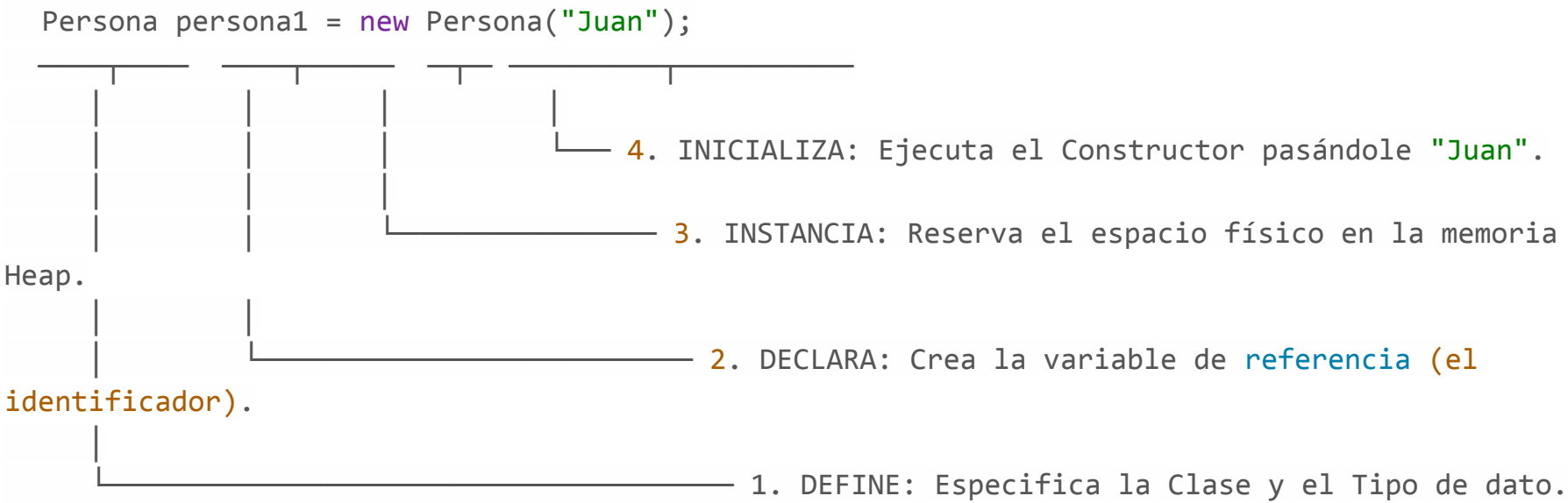
Tenemos:



¿Qué significa esta línea?

```
Persona persona1 = new Persona("Juan");
```

¿persona1 es el objeto?



¿Qué ocurre con new?

Tenemos:

```
Persona persona1 = new Persona("Juan");
```

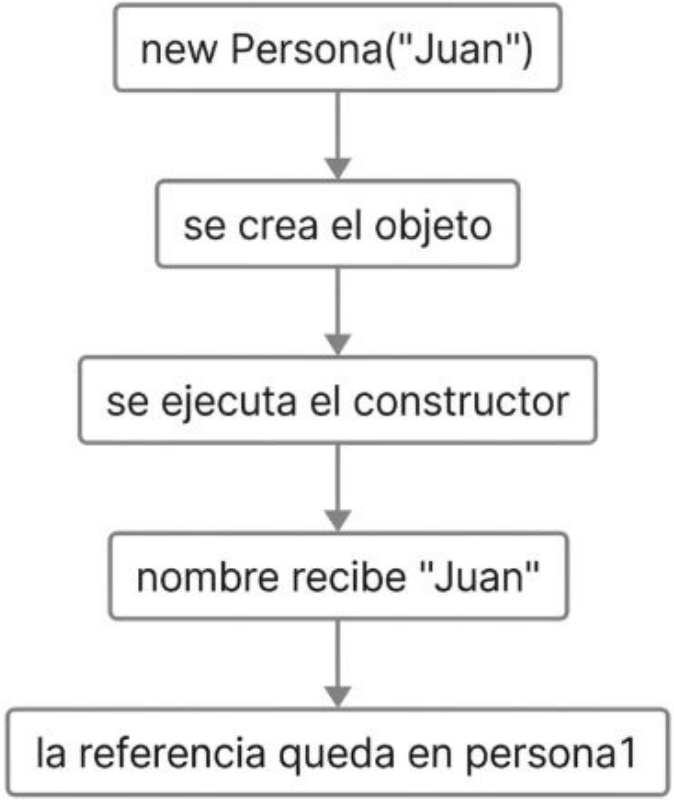
La expresión: `new Persona("Juan")`

provoca la creación de un objeto Persona

Pero además se ejecuta el constructor:

```
public Persona(String nombre)

    { this.nombre = nombre;
}
```



¿Dónde está el atributo?

En Persona.java:

```
public class Persona {
    private String nombre;

    public Persona(String
nombre) { this.nombre = nombre;
}

    public String getNombre() {
return nombre; }
}
```

¿Podemos hacer esto desde Main?

```
persona1.nombre = "Pedro";
```

El atributo está declarado como:

private

Por lo tanto, no puede accederse directamente desde Main.

¿Cómo accedemos al atributo?

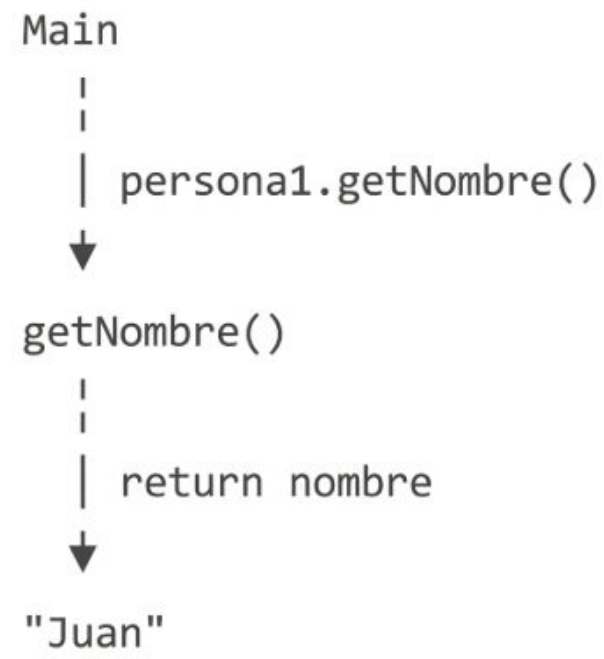
podemos ofrecer un método público:

```
public String getNombre()
    { return nombre; }
```

Y desde Main:

```
System.out.println(persona1.getNombre());
```

Recorrido



private: ¿qué problema resuelve?

Sin encapsulamiento

```
public String nombre;
```

Desde cualquier lugar podríamos hacer:

```
persona1.nombre = " ";
persona1.nombre = null;
persona1.nombre = "Juan";
```

La clase pierde control sobre su estado.

Con encapsulamiento

```
private String nombre;
```

y:

```
public String getNombre() {
    return nombre;
}
```

La clase controla cómo se accede a su estado.

Idea central

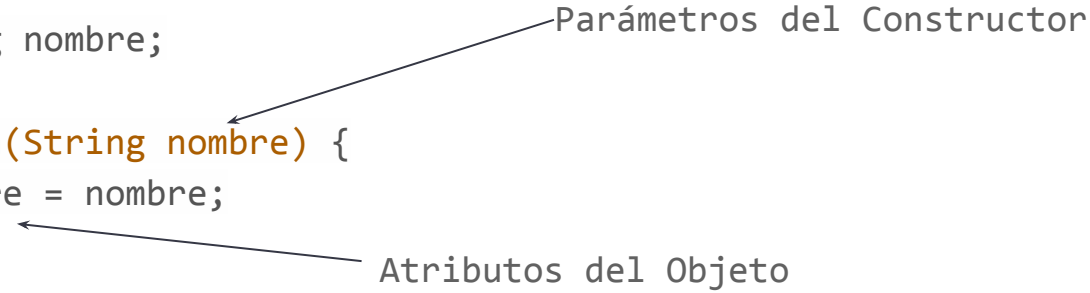
private protege el acceso directo al estado interno de un objeto.

¿Qué significa this.nombre = nombre?

Tenemos:

```
private String nombre;

public Persona(String nombre) {
    this.nombre = nombre;
}
```



Por eso:

```
this.nombre = nombre;
```

significa:

> "Asignar el valor del parámetro nombre al atributo nombre del objeto actual."

¿Qué pasa si escribimos nombre = nombre?

Supongamos:

```
private String nombre;

public Persona(String
nombre) {

    nombre = nombre;
}
```

¿Qué ocurre?

El parámetro se asigna a sí mismo.

nombre ← nombre

El atributo no recibe el valor.

Entonces:

`this.nombre`

significa:

"el atributo nombre del objeto actual".

Métodos: definir y utilizar

Definición

En Persona.java:

```
public String getNombre() {
    return nombre;
}
```

Acá estamos **definiendo** el método.

Llamada

En Main.java:

```
System.out.println(persona1.getNombre());
```

Acá estamos **invocando** el método.

¿Qué significa String antes del método?

Tenemos:

```
public String getNombre()
```

La palabra: String

es el **tipo de retorno** del método.

El método devuelve un String.

Por eso aparece:

```
return nombre;
```

Ejemplo

```
public String getNombre() {
    return nombre;
}
```

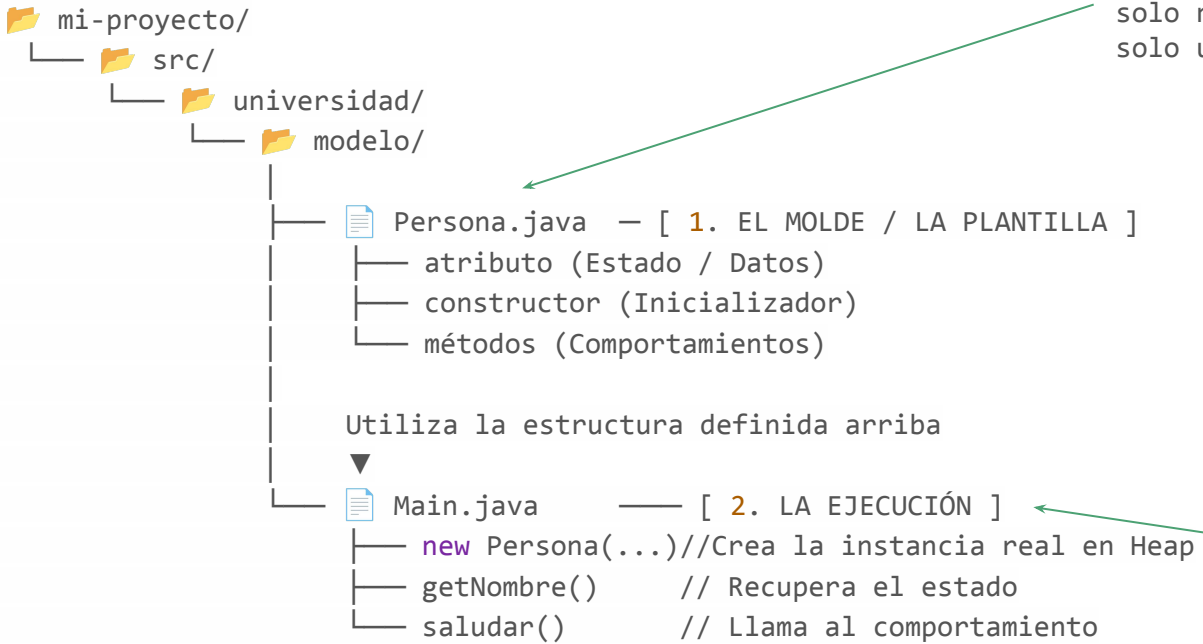
El resultado puede utilizarse así:

```
String nombre = persona1.getNombre();
```

o:

```
System.out.println(persona1.getNombre());
```

Mapa Conceptual



Es código estático. Define la anatomía del objeto, pero por sí solo no "existe" en la memoria. Es solo una plantilla en disco rígido.

Es código dinámico, lo ejecuta mediante la palabra clave new y da vida a las instancias reales que consumen memoria RAM y operan con datos concretos (como "Juan").

¿Qué pasa si Persona NO es public?

Podemos escribir:

```
class Persona {

}
```

sin poner public.

Esto se denomina acceso **package-private** o **default**.

Ejemplo

```
universidad.modelo
│
├── Persona.java
└── Alumno.java
```

Alumno puede acceder a Persona.

Pero:

```
universidad.main
│
└── Main.java
```

no puede utilizarla directamente como una clase accesible desde otro paquete.

Primer mapa de modificadores

Para una clase o miembro, podemos encontrarnos con:

private
public
protected
package-private

Pero **no significan lo mismo.**



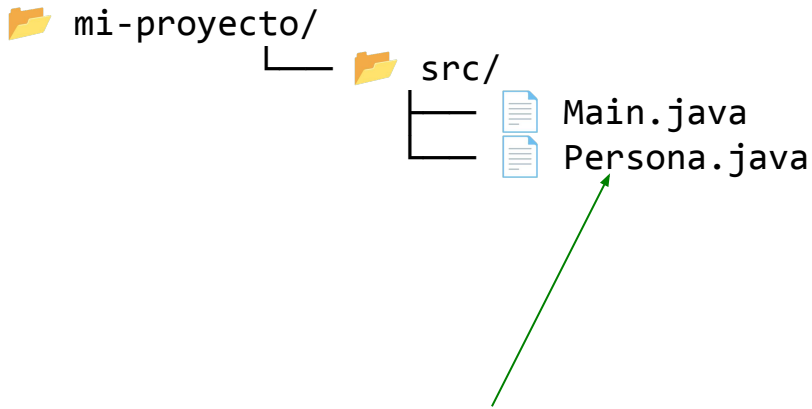
VISIBILIDAD DE LOS MODIFICADORES DE ACCESO EN JAVA

RESUMEN DEL NIVEL DE ACCESO Y ÁMBITO (ORDENADO POR RESTRICCIÓN)

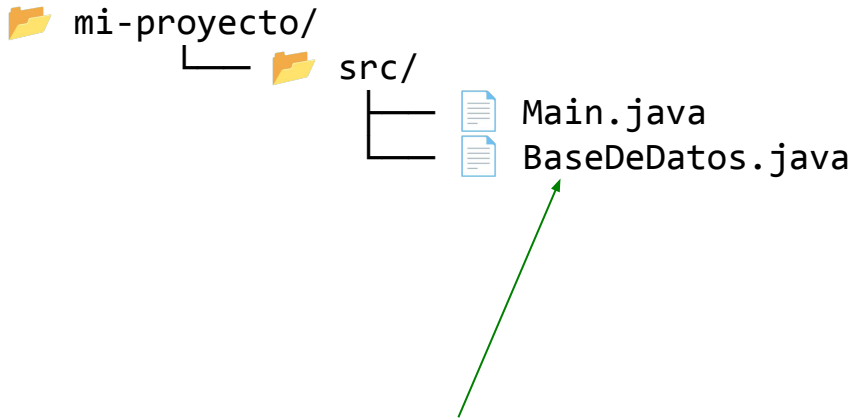
MODIFICADOR	Misma Clase?	Mismo Paquete?	Clases Hijas / Herencia?
[private]	SÍ (Solo interno)	NO	NO
[package-private] (Default)	SÍ	SÍ (Todo el paquete)	NO
[protected]	SÍ	SÍ	SÍ (Inclusive en otros paquetes)
[public]	SÍ	SÍ	SÍ (Acceso Total)

Comparación de acceso directo

¿Las clases también pueden ser private?




```java
// Archivo: Persona.java
private class Persona { //ERROR DE COMPILACIÓN
}
```




```java
public class BaseDeDatos {
  private class Conector { ..
  }
}
```

¿Y los constructores?

Los constructores también pueden tener modificadores de acceso.

```
public Persona(String nombre) {
    this.nombre = nombre;
}
```

public Flexibilidad: "Cualquiera puede fabricar este objeto en cualquier parte del sistema".

Podemos tener también:

```
private Persona() {
}
```

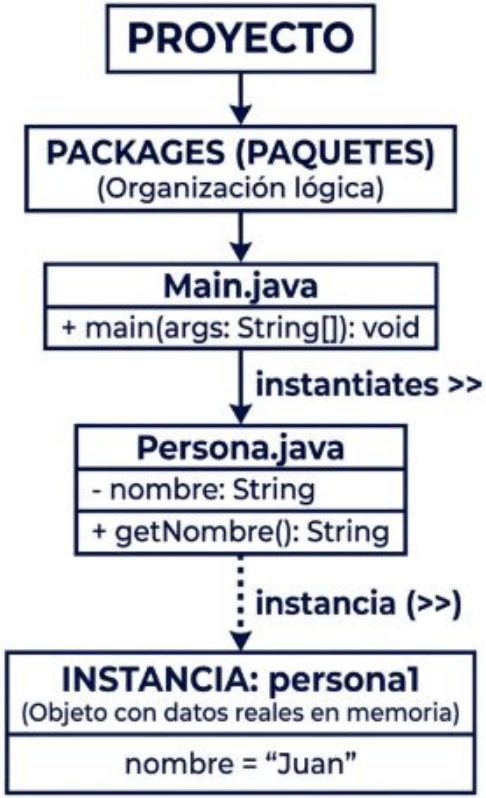
private Seguridad/Control: "Prohibido el new. Solo yo controlo cómo y cuándo se crea una instancia (Singleton / Clases Útiles)".

o:

```
protected Persona() {
}
```

protected Jerarquía: "Diseñado solo para que los hijos inicialicen a través de super(), blindando el objeto frente a clases externas".

¿Qué estamos construyendo?



¿Qué hacemos cuando tenemos clases relacionadas?

Por ejemplo:

```
📂 mi-proyecto/
    └── 📂 src/
        └── 📂 com/
            └── 📂 animales
                ├── 📄 Animal.java
                ├── 📄 Perro.java
                ├── 📄 Gato.java
                ├── 📄 Hamster.java
                └── 📄 Main.java
```

¿Tenemos que repetir todo el código común en cada clase?

El problema de copiar y pegar

Perro.java

```java
public class Perro {

    private String nombre;

    public Perro(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {

System.out.println("Comiendo");
    }
}
```

Y después:

Gato.java

```java
public class Gato {

    private String nombre;

    public Gato(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {

System.out.println("Comiendo");
    }
}
```

Una clase general: Animal

Animal.java

```java
package universidad.modelo;

public class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {
        System.out.println(nombre + " está comiendo");
    }
}
```

Ahora podemos crear una clase específica:

Perro.java

```java
package universidad.modelo;

public class Perro extends Animal {

}
```

¿Qué significa Perro extends Animal?

```
public class Perro extends Animal {..}
```

Se puede leer:

> Perro es un tipo de Animal.

La relación es:

Perro IS-A Animal

Ejemplos

Perro IS-A Animal
Gato IS-A Animal
Hamster IS-A Animal

No significa:

> "Perro copia y pega el código de Animal."

Significa que existe una **relación de herencia** entre ambos tipos.

¿Dónde se usa Perro?

Animal.java

```
public class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {
        System.out.println(nombre + " está comiendo");
    }
}
```

Perro.java

```
public class Perro extends Animal {..}
```

¿Dónde creamos el objeto?

En Main.java:

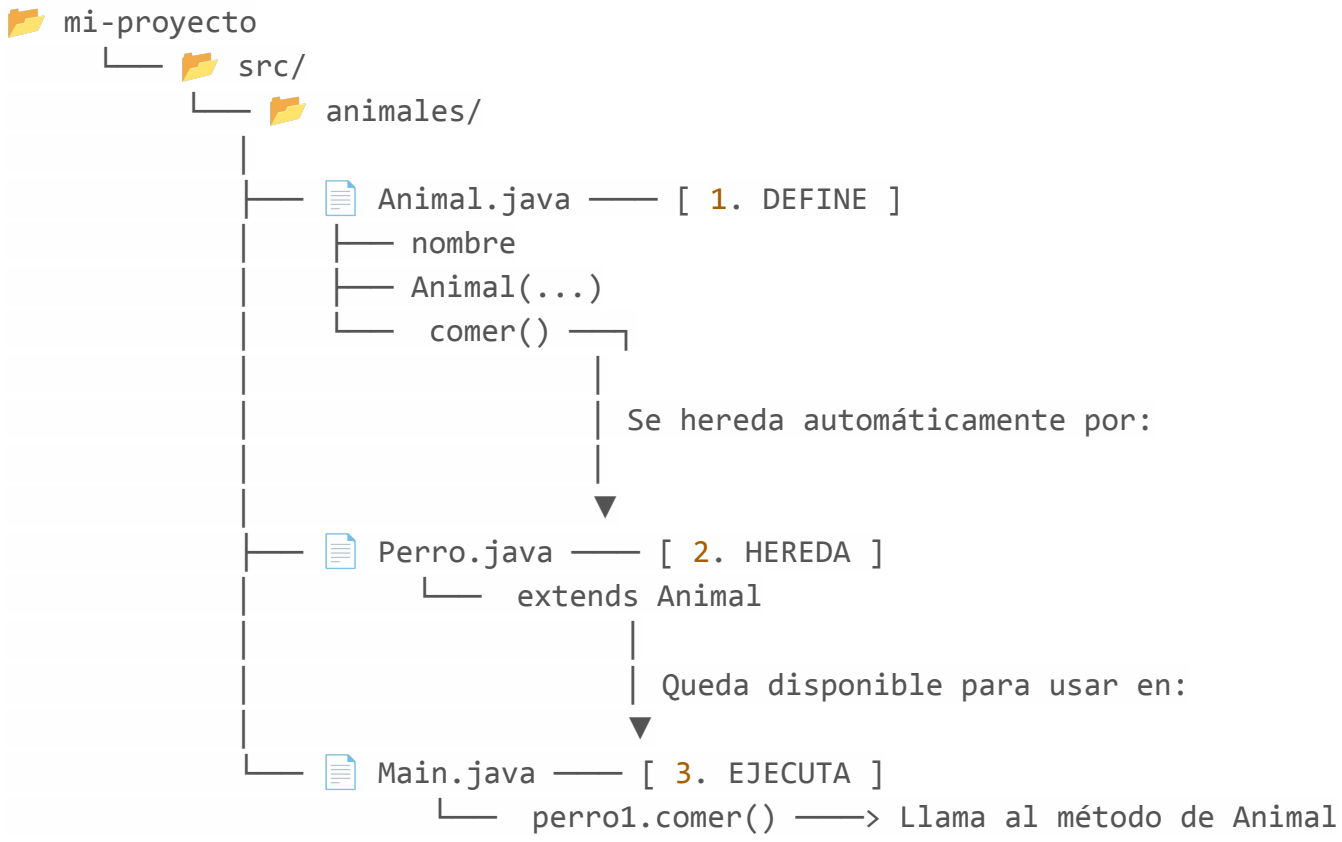
```
public class Main {

    public static void main(String[]
args) {

        Perro perro1 = new
Perro("Rex");

        perro1.comer();
    }
}
```

Recorrido de la herencia



¿Cómo se construye un Perro?

Tenemos:

Animal.java

```
package universidad.modelo;

public class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {
        System.out.println(nombre + " está comiendo");
    }
}
```

Perro.java

```
package universidad.modelo;

public class Perro extends Animal {

    public Perro(String nombre) {
        super(nombre);
    }
}
```

Y desde Main:

```
Perro perro1 = new Perro("Rex");
```

¿Qué constructor se ejecuta primero?

¿Qué es super(...)?

```
En Perro.java:
public Perro(String nombre) {
    super(nombre);
}
```

super(...) permite llamar al constructor de la **superclase**.

En este caso:

super(nombre);

llama a:

Animal(String nombre)

this(...) vs super(...)

#this(...)
#Llama a otro constructor de la misma clase.

```
public Persona() {
    this("Sin nombre");
}
```

#super(...)
#Llama al constructor de la clase padre.

```
public Perro(String nombre) {
    super(nombre);
}
```

Regla importante
Ambos deben aparecer como primera instrucción del constructor.
No podemos hacer:

```
public Perro(String nombre) {

    System.out.println("Creando perro");

    super(nombre); // ERROR
}
```

¿Qué pasa si no escribimos super()?

Supongamos:

Animal.java

```
public class Animal {

    public Animal(String nombre) {
        this.nombre = nombre;
    }
}
```

Perro.java

```
public class Perro extends Animal {

    public Perro(String nombre) {

    }
}
```

¿Compila?

Java intenta insertar implícitamente:

```
super();
```

Pero Animal **no tiene un constructor sin parámetros.**

Java intenta conceptualmente:

```
public Perro(String nombre) {
    super();
}
```

Pero solamente existe:

```
public Animal(String nombre)
```

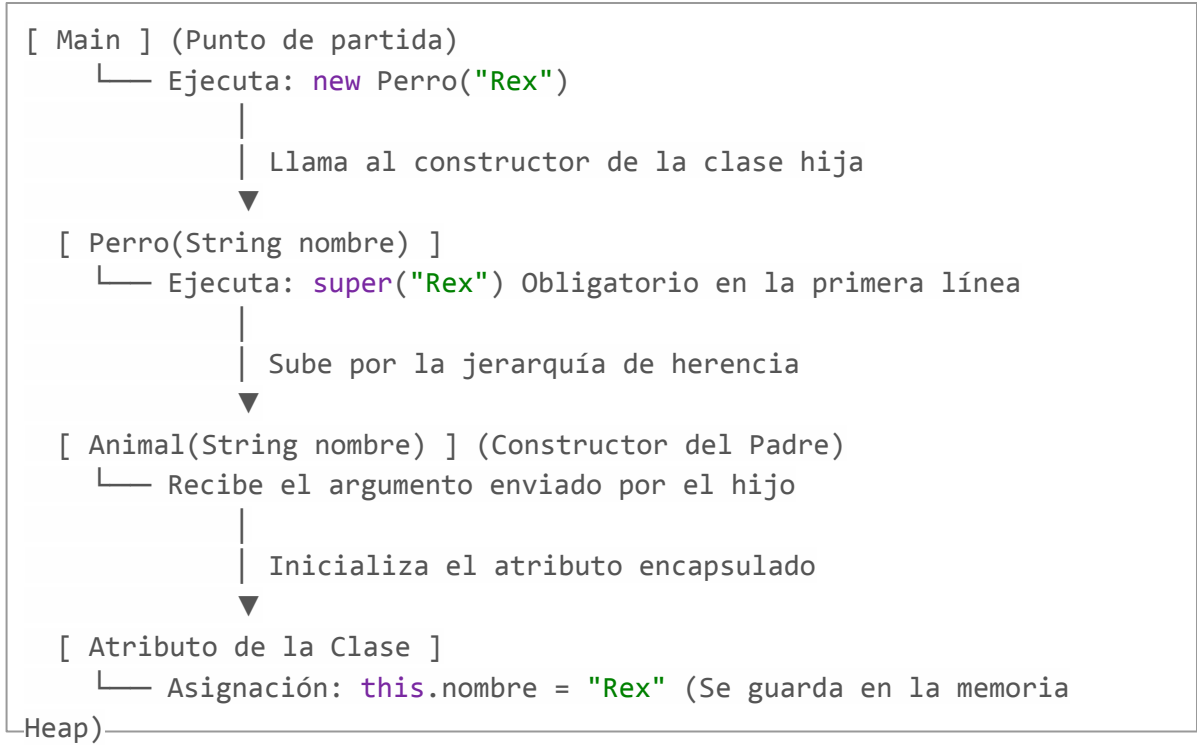
Solución

Si Animal tiene:

```
public Animal(String nombre) {
    this.nombre = nombre;
}
```

Entonces Perro debe indicar qué constructor quiere utilizar:

```
public Perro(String nombre) {
    super(nombre);
}
```



¿El atributo private se hereda?

Tenemos:

Animal.java

```
public class Animal {

    private String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }
}
```

¿Compila?

Perro.java

```
public class Perro extends
Animal {

    public void mostrarNombre()
{

System.out.println(nombre);
    }
}
```

Entonces, ¿cómo accede Perro al nombre?

Podemos mantener:

Animal.java

```
private String nombre;

public String getNombre() {
    return nombre;
}
```

Y desde Perro:

```
public void mostrarNombre() {
    System.out.println(getNombre());
}
```

Protección del Estado (`private`): El atributo `nombre` sigue bajo llave dentro de `Animal`. Ninguna clase externa (ni siquiera sus hijos) puede modificarlo directamente o corromper su valor.

Acceso Controlado (`getNombre`): La clase padre abre una "ventanilla pública" para entregar el dato de forma segura a quien lo pida amablemente.

Reutilización Inteligente: `Perro` no necesita romper las reglas de visibilidad; simplemente aprovecha que heredó el método `getNombre()` y lo invoca de forma transparente.

¿Y qué hace protected?

Animal.java

```java
public class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }
}
```

Perro.java

```java
public class Perro extends Animal {

    public void mostrarNombre() {
        System.out.println(nombre);
    }
}
```

Puerta Cerrada al Extraño: Si una clase externa cualquiera (que no sea hija y esté en otro paquete) intenta hacer `objeto.nombre`, Java le bloqueará el acceso por completo.

Puerta Abierta a la Familia: Al poner `protected`, la clase padre le dice a sus herederos: *"Confío en ustedes"*.

Acceso Directo: `Perro` ya no necesita pedirle permiso a un método intermedio como `getNombre()`. Puede leer y modificar la variable `nombre` directamente como si la hubiera escrito en su propio archivo.

Tabla de acceso

Modificador	Misma clase	Mismo paquete	Subclase	Cualquier clase
public	✅	✅	✅	✅
protected	✅	✅	✅	❌
default	✅	✅	⚠️*	❌
private	✅	❌	❌	❌

*** Mismo paquete:** Cualquier clase (sea hija o no) que comparta la misma carpeta física del proyecto puede leer y modificar el atributo directamente

ℹ️ **¿Quieres profundizar más?**

https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

¿public significa "todo se puede usar"?

No necesariamente.

Si tenemos:

```
public class Persona {

    private String nombre;

    public String getNombre() {
        return nombre;
    }
}
```

Desde otra clase podemos utilizar:

```
Persona p = new Persona("Juan");

p.getNombre();
```

Pero no:

```
p.nombre; // ERROR
```

¿Toda relación entre clases debe ser herencia?

Tenemos:

Perro IS-A Animal

Esto puede representarse mediante herencia.

Pero:

Auto HAS-A Motor

no significa que Auto sea un Motor.



¿Deberíamos hacer esto?

```
public class Auto extends Motor {
}
```

La relación conceptual no corresponde.

Ejemplo de HAS-A

Motor.java

```java
public class Motor {

    public void encender() {
        System.out.println("Motor encendido");
    }
}
```

Auto.java

```java
public class Auto {

    private Motor motor;

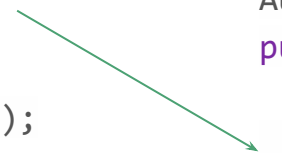
    public Auto() {
        motor = new Motor();
    }

    public void arrancar() {
        motor.encender();
    }
}
```

Main.java

```java
Auto auto1 = new Auto();

auto1.arrancar();
```





La Clase Object: ¿De dónde vienen las clases?

En Java existe una regla de oro: **Toda clase, por más simple que sea, hereda directa o indirectamente de una clase suprema llamada Object.**

Si tú no escribes extends, Java lo hace por ti tras bambalinas.

El árbol genealógico del código

Cuando creas una jerarquía, la estructura real en tu memoria se ve así:

```
Object (La raíz de todo en Java)
└── Animal (Hereda de Object automáticamente)
    └── Perro (Hereda de Animal, y por rebote, también de Object)
```

ℹ️ **¿Quieres profundizar más?**

Si te interesa entender a fondo todos los métodos ocultos que te regala esta clase suprema, dale un vistazo a la Guía de la Clase Object en Java. https://www.geeksforgeeks.org/java/object-class-in-java/

¿Podemos heredar de dos clases?

```
¿Podemos hacer esto?
public class Perro extends Animal, Mascota {
}
```

Java no permite que una clase herede directamente de dos clases.

Una clase puede extender **una sola clase.**

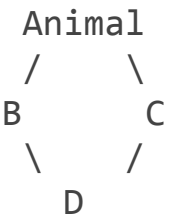
Pero...

Una clase sí puede implementar múltiples interfaces.

```
public class Perro extends Animal
                    implements Nadador, Corredor {
}
```

El problema del diamante

Imaginemos:



Si Java permitiera herencia múltiple de clases:

```
class D extends B, C
```

podríamos preguntarnos:

¿Qué implementación debería utilizar D?

```
Animal.metodo()
      ↑
   ¿B o C?
```

Java evita este problema no permitiendo herencia múltiple de clases.

¿Y qué pasa si queremos definir capacidades?

Supongamos que tenemos:

Perro
Ave
Pez

y queremos representar la capacidad:

NADAR

No necesariamente todos son de la misma familia de herencia.

Podemos definir una interfaz:

Nadador.java

```java
public interface Nadador {

    void nadar();
}
```

¿Cómo implementamos una interfaz?

Nadador.java

```java
public interface Nadador {

    void nadar();
}
```

Pez.java

```java
public class Pez implements Nadador {

    @Override
    public void nadar() {
        System.out.println("El pez está nadando");
    }
}
```

Main.java

```java
Pez pez1 = new Pez();

pez1.nadar();
```

El Flujo del Contrato (Diferencia Fundamental)

interface (El Contrato)
No hace ningún trabajo real.
Solo declara la lista de tareas vacías (Ejemplo: void nadar();).

Se firma el contrato usando la palabra clave: implements

class (Quien cumple el contrato)
Firma la interfaz y se hace cargo.
Está obligada por ley a escribir el código real dentro del método.

¿Una clase puede implementar varias interfaces?

Por ejemplo:

```
public class Pato implements Nadador, Volador {

    @Override
    public void nadar() {
        System.out.println("Nadando");
    }

    @Override
    public void volar() {
        System.out.println("Volando");
    }
}
```

El caso del Pato:

Un pato es una sola criatura, pero cumple dos contratos totalmente independientes en el mundo real:

Rol 1: Es un Nadador ——> Está obligado a implementar el método nadar().

Rol 2: Es un Volador ——> Está obligado a implementar el método volar().

Clase abstracta: ¿para qué sirve?

1. [LA HIPÓTESIS]
   - Supongamos la jerarquía: Animal ——> Perro, Gato, Pez
   - Queremos una clase general Animal para unificar el código.

2. [EL QUIEBRE: Pero...]
   - ¿Tiene sentido crear un Animal genérico?
   - Animal animal1 = new Animal(...);
   - Quizás no. En la realidad no existe un "Animal" a secas.

3. [EL OBJETIVO]
   - Queremos que Animal represente una idea general (una plantilla).
   - Solución: Hacerla [abstract class Animal] para prohibir el "new".

¿Podemos hacer new Animal()?

Si tenemos:

```
public abstract class Animal {
}
```

intentamos:

```
Animal animal1 = new Animal(); // ERROR
```

Una clase abstracta **no puede ser instanciada directamente.**

Sí podemos:

```
Perro perro1 = new Perro("Rex");
```

si:

```
public class Perro extends Animal {
}
```

La clase abstracta sirve para definir aquello que es común o general.

Las subclases concretas representan objetos que sí podemos crear.

Una clase abstracta puede tener métodos concretos

Animal.java

```java
public abstract class Animal {

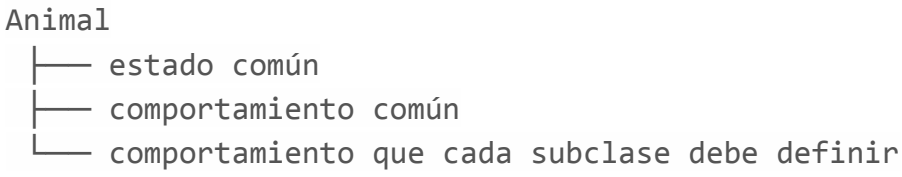
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void comer() {
        System.out.println(nombre + " está comiendo");
    }
}
```

Una clase abstracta puede contener comportamiento completamente implementado.

Esto es justamente una de sus ventajas.



¿Qué es un método abstracto?

Podemos escribir:

```
public abstract class Animal {

    public abstract void hacerSonido();
}
```

El método:
`public abstract void hacerSonido();`
no tiene cuerpo.
No escribimos:

```
{
    ...
}
```

Estamos diciendo:

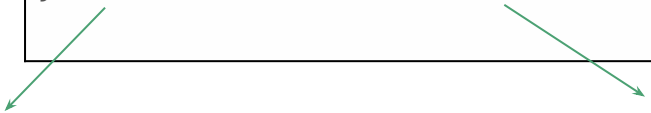
> Toda clase concreta que herede de Animal deberá definir cómo realiza hacerSonido().

¿Quién implementa el método abstracto?

Animal.java

```java
public abstract class Animal {

    public abstract void hacerSonido();
}
```



Perro.java

```java
public class Perro extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

Gato.java

```java
public class Gato extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Miau");
    }
}
```

¿Qué pasa si Perro no implementa el método?

Tenemos:

```
public abstract class Animal {

    public abstract void hacerSonido();
}
```

Y:

```
public class Perro extends Animal {

}
```

¿Compila?

Regla:

Una clase concreta debe implementar todos los métodos abstractos heredados.

Dos opciones

Opción 1

Implementar:

```
@Override
public void hacerSonido() {
    System.out.println("Guau");
}
```

Opción 2

Hacer que Perro también sea abstracta:

```
public abstract class Perro extends Animal {
}
```

¿Una clase abstracta puede tener constructor?

Animal.java

```java
public abstract class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public abstract void hacerSonido();
}
```

Aunque no podamos hacer:

```java
new Animal("Animal");
```

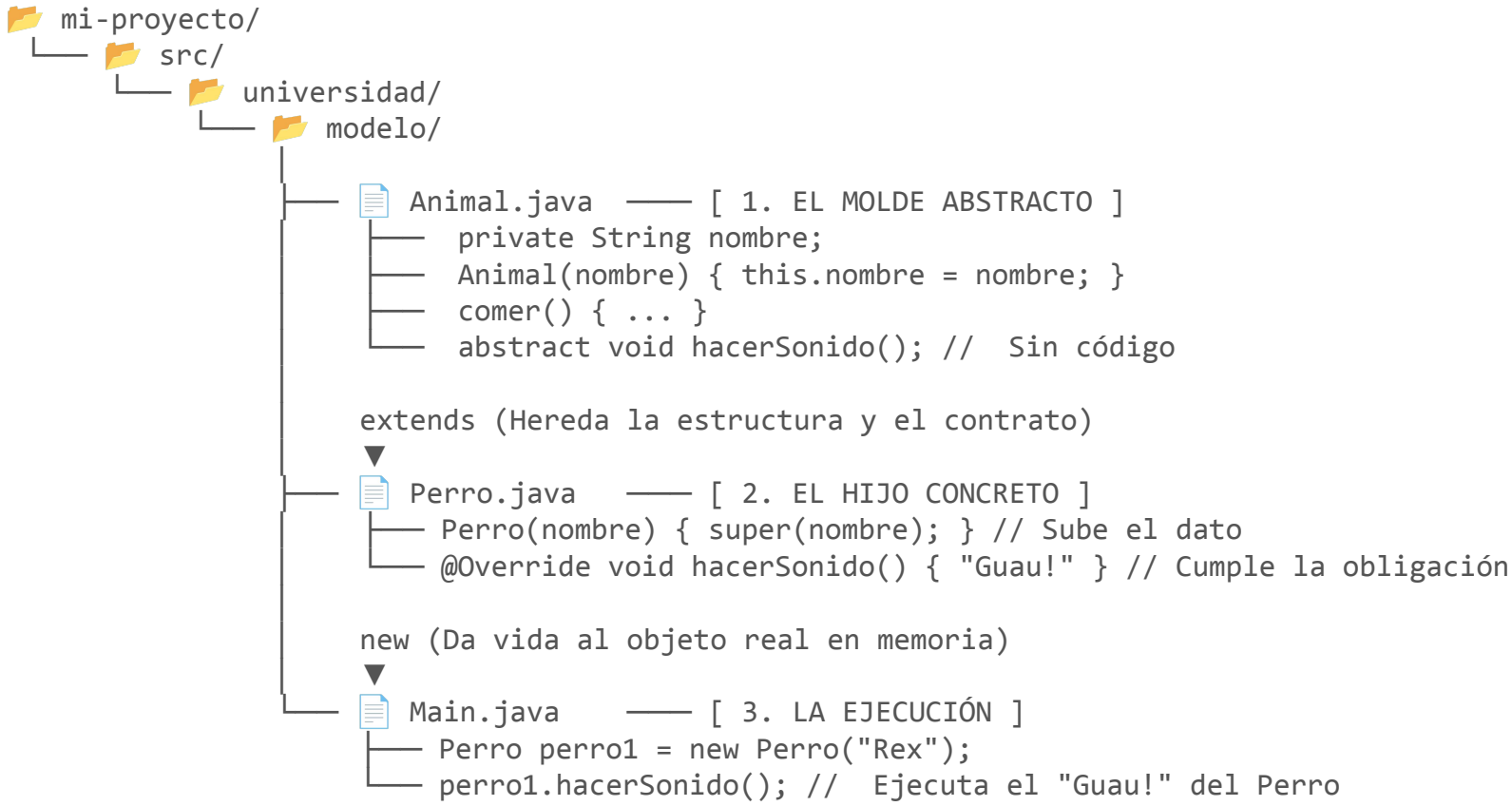
sí podemos ejecutar su constructor al crear una subclase:

```java
new Perro("Rex");
```

mediante:

```java
super("Rex");
```

Recorrido completo de una clase abstracta



Clase abstracta vs interfaz

Clase abstracta

"Es una base común para tipos relacionados."

```
public abstract class Animal {

    protected String nombre;

    public Animal(String nombre)
{
        this.nombre = nombre;
    }
      public void comer() {

System.out.println("Comiendo");
    }
      public abstract void
hacerSonido();
    }
```

Interfaz

"Define un contrato o capacidad que una clase puede cumplir."

```
public interface Nadador {
    void nadar();
}
```

BIBLIOGRAFÍA

Bibliografía y fuentes de consulta

- **Aguilar, L. J.** — *Java: Programación Orientada a Objetos*.
- **Martínez, I. Z., Aguilar, L. J.** — *Estructuras de Datos en Java*.
- **Oracle** — *Java Documentation*. Documentación oficial del lenguaje Java.
- **Material teórico y práctico de la cátedra** — Teorías y prácticas previas de Programación Orientada a Objetos.